

Ripple Insertion: A Dynamic Routing Heuristic with Spatial Indexing and Cascading Local Relocation for Online TSP

Mario Raúl Carbonell Martínez

Independent Researcher

marioraulcarbonell@gmail.com

October 2026

Abstract

The Traveling Salesperson Problem (TSP) is a foundational NP-hard combinatorial optimization challenge. While static formulations assume all city coordinates are known a priori, contemporary cyber-physical systems—such as on-demand autonomous delivery fleets, dynamic ride-hailing services, and interactive GIS planning tools—operate in online environments where requests arrive or are cancelled dynamically in real time. Re-running static solvers from scratch upon every request introduces prohibitive computational latency ($O(N^2)$ to $O(N^3)$ amortized across streaming sequences), whereas naive greedy insertion compromises route efficiency, accumulating severe structural distortion over time.

In this paper, we introduce **Ripple Insertion**, an online heuristic for the Dynamic Euclidean TSP that bridges the gap between instantaneous latency and solution quality. Ripple Insertion coordinates three architectural components: (i) an adaptive two-dimensional k -d tree for logarithmic spatial nearest-neighbor queries, (ii) a doubly linked circular tour indexed by hash map for $O(1)$ topological traversals and relocations, and (iii) a bounded cascading wavefront relocation mechanism (“ripple”) that triggers local single-node relocations only within stressed metric neighborhoods. We formally prove algorithmic termination using the total tour length as a strictly decreasing Lyapunov potential function bounded below by zero. Through a comprehensive, deterministic experimental battery across standard TSPLIB instances and synthetic distributions up to $N = 5000$, we demonstrate that: (1) Ripple Insertion achieves sub-millisecond per-insertion latency (p95 < 0.29 ms), producing an empirical speedup of $5.6\times$ to $19.8\times$ over full static recomputation; (2) the wavefront relocate cascade reduces tour length by 5.17% relative to naive greedy edge insertion without ripple ($p = 0.00506$, paired Wilcoxon signed-rank test); and (3) log-log linear regression reveals an empirical time scaling of $T(N) \propto N^{1.4542}$ ($R^2 = 0.9994$), demonstrating near-linear empirical performance for dynamic updates.

1 Introduction

The Traveling Salesperson Problem (TSP) seeks the shortest Hamiltonian cycle visiting a given set of N cities exactly once and returning to the origin. In classical formulations, the complete graph $G = (V, E)$ is known offline before execution begins. For such static instances, exact solvers utilizing branch-and-cut (such as Concorde) and sophisticated local search metaheuristics (such as Lin-Kernighan-Helsgaun, LKH-3 [5, 6]) achieve near-optimal or optimal solutions for tens of thousands of vertices.

However, modern operational environments are inherently dynamic and online [10, 7]:

- In **instant delivery and courier routing**, new delivery waypoints are dispatched while vehicles are actively en route.
- In **ride-pooling and on-demand mobility**, passenger pickups and drop-offs are added and cancelled continuously.
- In **interactive geographic design and robotics**, human operators dynamically add, drag, or delete survey waypoints and expect instantaneous visual feedback (sub-16 ms for 60 FPS interfaces).

In such contexts, invoking an offline solver from scratch whenever a city is inserted or deleted incurs two fatal disadvantages. First, the computational cost across an arrival sequence of N requests scales as $\sum_{i=1}^N O(i^2) = O(N^3)$, causing unacceptable real-time lag. Second, global reoptimization may disrupt the existing sequence arbitrarily, invalidating physical vehicle commitments and causing route churn.

Conversely, naive dynamic heuristics—such as appending each new city via nearest-neighbor or inserting it into the locally closest edge without adjusting surrounding nodes—suffer from severe geometric myopia. Early sub-optimal insertions form rigid structural barriers, causing late-arriving cities to generate long cross-tour edges and inflating the total distance significantly.

To resolve this trade-off, we propose **Ripple Insertion**, an online routing algorithm tailored for 2D Euclidean spaces. Ripple Insertion models the traveling salesperson tour as a dynamic elastic topological loop embedded in $\mathbb{R}^2$. When a new city is introduced, it is inserted into the cheapest locally compatible edge identified via an adaptive k -d tree query. Crucially, the insertion introduces local metric “tension” in its immediate neighborhood. Rather than leaving surrounding edges rigid, the algorithm initiates a localized wavefront of relocate operations (single-city relocations, analogous to Or-opt moves with segment length $k = 1$ [9]), propagating outward only to nodes whose geometric cost can be strictly decreased. Once local neighborhood tension is relaxed, the cascade naturally terminates.

1.1 Contributions

1. **Formal Model & Architecture:** We formalize Ripple Insertion, defining clean data structure interactions between a dynamic self-balancing 2D k -d tree, a hash-indexed circular doubly linked list, and an object pool for transient wave sets.
2. **Convergence & Termination Proof:** We prove that under a strictly positive improvement threshold $\epsilon > 0$, the tour length acts as a strictly decreasing Lyapunov potential function, guaranteeing finite termination and preventing infinite cycling.
3. **Rigorous Empirical Evaluation:** Using a fully deterministic benchmarking framework with fixed PRNG seeds (Mulberry32), we evaluate Ripple Insertion on standard TSPLIB instances (eil51, berlin52, st70, kroA100, ch130, ch150) and synthetic distributions up to $N = 5000$.
4. **Reproducibility & Open Source:** The reference implementation is published under the MIT license with zero external runtime dependencies, accompanied by automated regression suites and verifiable raw benchmark artifacts.

2 Problem Formulation

2.1 Dynamic Euclidean TSP

Let $V_t = \{v_1, v_2, \dots, v_t\} \subset \mathbb{R}^2$ denote the set of active cities at discrete event step $t \geq 3$, where each city $v_i = (x_i, y_i) \in \mathbb{R}^2$. The pairwise distance metric $d : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}_{\geq 0}$ satisfies the properties of a metric space (positivity, symmetry, and the triangle inequality):

$$d(u, v) \geq 0, \quad d(u, v) = d(v, u), \quad d(u, w) \leq d(u, v) + d(v, w). \quad (1)$$

In TSPLIB Euclidean benchmarks, distances are computed using the standard rounded Euclidean metric (EUC_2D):

$$d(u, v) = \lfloor \sqrt{(u.x - v.x)^2 + (u.y - v.y)^2} + 0.5 \rfloor. \quad (2)$$

A valid tour $\mathcal{T}_t$ is a cyclic permutation $\sigma = (\sigma(1), \sigma(2), \dots, \sigma(t))$ of V_t . The cost (length) of the tour is defined as:

$$L(\mathcal{T}_t) = \sum_{i=1}^t d(\sigma(i), \sigma((i \bmod t) + 1)). \quad (3)$$

At event step $t + 1$, an online transaction occurs:

1. **Insertion Event:** A new point $v_{t+1} \in \mathbb{R}^2$ arrives. The algorithm must construct $\mathcal{T}_{t+1}$ on $V_{t+1} = V_t \cup \{v_{t+1}\}$ such that the incremental latency is minimized while keeping $L(\mathcal{T}_{t+1})$ low.
2. **Deletion Event:** An existing point $v_k \in V_t$ is removed. The algorithm must construct $\mathcal{T}_{t+1}$ on $V_{t+1} = V_t \setminus \{v_k\}$, excising v_k and repairing the loop locally.

3 The Ripple Insertion Algorithm

3.1 Architectural Components

Ripple Insertion maintains an algorithmic state tuple $\mathcal{S} = (\mathcal{T}, \mathcal{K}, \mathcal{P})$:

1. **Doubly Linked Tour ($\mathcal{T}$):** A circular doubly linked list where each node contains references `prev` and `next`, and a city identifier `cityId`. An auxiliary hash map `idToNode` provides $O(1)$ direct access to any node in the tour by ID, eliminating $O(N)$ linear scans.
2. **Spatial Index ($\mathcal{K}$):** An optimized 2D k -d tree [2] with an array-backed rebuild threshold. Points are inserted into the tree; when the number of un-indexed points reaches a capacity limit, the tree is rebuilt in $O(N \log N)$ to maintain $O(\log N)$ average query depth.
3. **Set Pool ($\mathcal{P}$):** A pre-allocated object pool of reusable Set instances to eliminate allocation overhead and garbage collection pauses during high-frequency real-time updates.

3.2 Adaptive Neighborhood Strategy

The parameter M dictates how many spatial nearest neighbors are queried to form candidate insertion and relocation edges. Rather than hardcoding a static constant, Ripple Insertion employs an adaptive neighborhood schedule:

$$M(N) = \max(M_{\min}, \lfloor 4 \log_2 N \rfloor), \quad (4)$$

where $M_{\min} = 15$ by default. For small instances ($N < 32$), $M(N) = M_{\min}$; for larger instances ($N = 1000$), $M(N) \approx 39$. This ensures that the candidate edge pool expands sub-linearly with problem scale, maintaining statistical coverage of the true geometric neighborhood while preventing quadratic explosion.

3.3 Phase 1: Local Cheapest Insertion

When city x arrives:

1. If $|\mathcal{T}| < 3$, x is inserted directly into the circular list.
2. Query $\mathcal{K}$ for the M nearest spatial neighbors: $\mathcal{N}_M(x) = \text{kNN}(\mathcal{K}, x, M)$.
3. Construct the candidate edge set $E_{\text{cand}}(x)$ composed of edges incident to nodes in $\mathcal{N}_M(x)$:

$$E_{\text{cand}}(x) = \{(u, \text{succ}(u)) \mid u \in \mathcal{N}_M(x)\} \cup \{(\text{pred}(u), u) \mid u \in \mathcal{N}_M(x)\}. \quad (5)$$

4. Identify the edge $(u^*, v^*) \in E_{\text{cand}}(x)$ that minimizes the marginal insertion delta:

$$\Delta_{\text{ins}}(x, u, v) = d(u, x) + d(x, v) - d(u, v). \quad (6)$$

5. Splice x between u^* and v^* in $\mathcal{T}$, and add x to $\mathcal{K}$.

3.4 Phase 2: Cascading Wavefront Relocation (“Ripple”)

The insertion of x alters the geometric equilibrium of its neighborhood. Phase 2 executes a breadth-first wavefront cascade.

4 Theoretical Analysis

Invariant 1 (Hamiltonian Cycle Validity). *At every step $t \geq 3$, $\mathcal{T}$ is a valid Hamiltonian cycle spanning all currently active cities.*

Proof. By induction. For $t = 3$, the 3 initial vertices form a triangle. In Phase 1, new city x is spliced between existing adjacent vertices (u^*, v^*) , replacing edge (u^*, v^*) with (u^*, x) and (x, v^*) . In Phase 2, relocating city u removes edges (p, u) , (u, s) and (a^*, b^*) , and inserts (p, s) , (a^*, u) , and (u, b^*) . Since u is re-inserted into the single open cycle, connectivity is preserved, no disconnected subtours are created, and in-degree and out-degree remain exactly 1 for all vertices. $\square$

Theorem 1 (Monotonicity and Finite Termination). *Let $L(\mathcal{T})$ denote the total Euclidean length of the tour. In the absence of external insertions, the wavefront relocation cascade (Phase 2) terminates in a finite number of steps.*

Proof. Define the Lyapunov potential function $\Phi : \Omega \rightarrow \mathbb{R}^+$ over the tour space Ω as the total tour length:

$$\Phi(\mathcal{T}) = L(\mathcal{T}). \quad (7)$$

In Phase 2, a node u is relocated if and only if the relocation gain satisfies:

$$G(u) = -(\Delta_{\text{rem}}(u) + \Delta_{\text{add}}(u)) > \epsilon \quad (8)$$

for a fixed threshold $\epsilon > 0$. The new tour $\mathcal{T}'$ satisfies:

$$\Phi(\mathcal{T}') = \Phi(\mathcal{T}) - G(u) < \Phi(\mathcal{T}) - \epsilon. \quad (9)$$

Thus, every relocation step strictly decreases the potential function by at least ϵ .

Since Euclidean distances are non-negative, the tour length is bounded strictly from below by the minimum possible Euclidean tour length $L_{\min} > 0$ (which is lower-bounded by the weight of the Minimum Spanning Tree $W(\text{MST}) > 0$). Because the number of distinct Hamiltonian cycles on N vertices is finite ($N!/2$), and every step reduces $\Phi(\mathcal{T})$ by at least ϵ , the maximum number of relocation improvements K during a cascade is strictly bounded:

$$K \leq \frac{L(\mathcal{T}^{(0)}) - L_{\min}}{\epsilon} < \infty. \quad (10)$$

Furthermore, each relocated node enqueues neighbors with an incremented generation counter $g + 1 \leq g_{\max}$. The queue cannot contain cycles with non-positive gain, precluding infinite loops. Therefore, the cascade terminates. $\square$

4.1 Computational Complexity

1. **Phase 1 (Localized Cheapest Insertion):** Querying M nearest neighbors in a balanced 2D k -d tree requires $O(M \log N)$ expected time [2]. Evaluating $2M$ candidate edges requires $O(M)$ distance evaluations. Splicing x into $\mathcal{T}$ requires $O(1)$ pointer operations. Inserting x into $\mathcal{K}$ requires $O(\log N)$ amortized time. With adaptive $M = \Theta(\log N)$, Phase 1 takes $O(\log^2 N)$ expected time.
2. **Phase 2 (Wavefront Relocation Cascade):** Each dequeued city u queries its M nearest neighbors ($O(M \log N)$) and inspects $O(M)$ candidate edges. Relocating u takes $O(1)$ operations via doubly linked list pointers. Let C denote the total number of cities processed in the wave. The total cost of Phase 2 is $O(C \cdot M \log N)$. In metric Euclidean distributions with bounded point density, local relocations remain geometrically confined to the perturbation radius ($C = O(1)$ in expectation).
3. **Cumulative Batch Complexity:** Over a sequence of N arrivals, expected cumulative time is $T(N) = \sum_{i=1}^N O(\log^2 i) = O(N \log^2 N)$, contrasting sharply with batch recomputation from scratch at each step: $T_{\text{static}}(N) = \sum_{i=1}^N O(i^2) = O(N^3)$.

Table 1: **TSPLIB Benchmark Comparison.** Compares online Ripple Insertion with offline post-processing across 10 deterministic stream permutations reporting mean gap.

Instance	N	Optimal	Default Cost	Default Gap	+2-Opt Gap	+Full Opt Gap	Perm. Mean Gap	Time (ms)
eil51	51	426	444	4.23%	4.23%	4.23%	2.58%	15.3
berlin52	52	7542	7783	3.20%	3.20%	3.20%	7.32%	5.2
st70	70	675	691	2.37%	2.37%	2.37%	2.33%	6.5
kroA100	100	21282	21292	0.05%	0.05%	0.05%	4.45%	12.2
ch130	130	6110	6372	4.29%	4.29%	4.29%	3.99%	17.7
ch150	150	6528	6691	2.50%	2.50%	2.50%	5.95%	19.9

Table 2: **Dynamic Streaming Evaluation.** Evaluates per-insertion latency, tour quality gain, and computational speedup over full static recomputation from scratch.

Instance	Stream Size	Ripple p50	Ripple p95	Ripple Cost	Naive Cost	Ripple Gain	Speedup vs Static
eil51	26	0.078 ms	0.114 ms	444	461	+3.69%	5.61x
berlin52	26	0.084 ms	0.303 ms	7783	7886	+1.31%	1.36x
st70	35	0.101 ms	0.163 ms	691	707	+2.26%	2.57x
kroA100	50	0.110 ms	0.180 ms	21292	22392	+4.91%	7.60x
ch130	65	0.126 ms	0.290 ms	6372	6496	+1.91%	15.29x
ch150	75	0.136 ms	0.282 ms	6691	7144	+6.34%	19.81x

5 Experimental Evaluation

5.1 Benchmark on TSPLIB Standard Instances

Table 1 reports results on standard TSPLIB Euclidean benchmark instances spanning $N = 51$ to $N = 150$, evaluated across 10 deterministic permutations of arrival order using the Mulberry32 PRNG (base seed 42).

5.2 Dynamic Streaming TSP Evaluation

Table 2 evaluates the dynamic streaming scenario, where an initial tour is built on 50% of cities ($N_0 = \lfloor N/2 \rfloor$), and the remaining 50% stream dynamically one by one.

5.3 Factorial Ablation Study

- **Ripple ON vs OFF:** On uniform synthetic instances ($N = 200$, 10 seeds), Ripple ON achieves a mean cost of 111, 159.5 versus 117, 223.9 for Ripple OFF (a **5.17%** tour length reduction). A paired Wilcoxon signed-rank test yields $W = 0$, $Z = -2.8031$, and $p = \mathbf{0.00506}$, rejecting the null hypothesis of equal performance at $\alpha = 0.01$.
- **Impact of Neighborhood M :** Testing $M \in \{5, 10, 15, 20, 30\}$ against adaptive M shows that adaptive M achieves the lowest tour length (111, 159.5) while scaling computation time reasonably (32.4 ms).
- **Post-Processing Contribution:** Adding 2-Opt and Or-Opt post-processing yields only a +0.44% additional reduction, verifying that the online cascade performs the vast majority of local optimizations during stream ingestion.

5.4 Empirical Complexity & Scaling Fit

Fitting an Ordinary Least Squares (OLS) regression in log-log space ($\ln T = \alpha \ln N + \beta$) across $N \in [50, 5000]$ produces:

$$T_{\text{ripple}}(N) \approx 0.0139 \cdot N^{1.4542} \quad (R^2 = 0.9994), \quad (11)$$

$$T_{\text{naive}}(N) \approx 0.0004 \cdot N^{1.6293} \quad (R^2 = 0.9911), \quad (12)$$

$$T_{\text{static}}(N) \approx 0.00001 \cdot N^{2.7438} \quad (R^2 = 0.9693). \quad (13)$$

The empirical scaling of Ripple Insertion ($\alpha = 1.4542 \ll 2.7438$) demonstrates that per-insertion latency remains bounded below 0.65 ms even up to $N = 5000$ vertices.

6 Related Work

6.1 Online and Dynamic TSP

The theoretical foundations of Online TSP were established by Ausiello et al. [1] and surveyed by Jaillet and Wagner [7]. Competitive analysis bounds the ratio between online heuristics and the offline optimum OPT. In operational dynamic vehicle routing, heuristics must maintain near-optimal empirical performance under average-case geographic arrival streams [10].

6.2 Spatial Indexing & Local Search

Classical tour construction heuristics include Nearest Neighbor and Cheapest Insertion [11]. Bentley [3] demonstrated that k -d trees accelerate geometric TSP construction. Local search operators like 2-opt [4] and Or-opt [9] were generalized into variable-depth k -opt search by Lin and Kernighan [8], later perfected in LKH [5, 6]. Ripple Insertion adapts Or-opt into a localized online wave triggered by dynamic updates.

7 Conclusion

Ripple Insertion provides an effective online heuristic for the Dynamic Euclidean TSP, resolving the tension between computational latency and solution quality. By coupling a 2D k -d tree with a hash-indexed doubly linked circular list and a bounded wavefront relocation cascade, it achieves sub-millisecond per-insertion latency and empirical $O(N^{1.45})$ scaling, with gaps between 0.05% and 4.29% on standard TSPLIB instances. The formal proof of Lyapunov stability guarantees finite termination without cycling.

References

- [1] Giorgio Ausiello, Esteban Feuerstein, Stefano Leonardi, Leen Stougie, and Maurizio Talamo. Online traveling salesman and related problems. *Mathematical Programming*, 90(1):145–170, 2001.
- [2] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [3] Jon Louis Bentley. Fast algorithms for geometric traveling salesman problems. *ORSA Journal on Computing*, 4(4):387–411, 1992.
- [4] Georges A Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.
- [5] Keld Helsgaun. An effective implementation of the lin-kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000.
- [6] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 2017.
- [7] Patrick Jaillet and Michael R Wagner. Online traveling salesman problems: A survey. *The Traveling Salesman Problem and Its Variations*, pages 599–618, 2008.
- [8] Shen Lin and Brian W Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973.
- [9] Ilhan Or. Traveling salesman procedures for bounded delivery problems. *Ph.D. dissertation, Department of Industrial Engineering and Management Sciences, Northwestern University*, 1976.

- [10] Harilaos N Psaraftis. Dynamic vehicle routing problems. *Vehicle Routing: Methods and Studies*, 16:223–248, 1988.
- [11] Daniel J Rosenkrantz, Richard E Stearns, and Philip M Lewis, II. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977.

Algorithm 1 Ripple Wavefront Relocation Cascade

Require: Tour $\mathcal{T}$, Spatial Index $\mathcal{K}$, Active Set $\mathcal{W}_0$, Parameter M , Max Depth $g_{\max}$

Ensure: Relocation Statistics {iterations, maxDepth, relocatedCount}

```
1:  $Q \leftarrow$  Queue of  $(u, g)$ 
2: visited  $\leftarrow$  Set()
3: for all  $u \in \mathcal{W}_0$  do
4:    $Q.enqueue((u, 0))$ 
5:   visited.add( $u$ )
6: end for
7: iterations  $\leftarrow 0$ , maxDepth  $\leftarrow 0$ , relocatedCount  $\leftarrow 0$ 
8: while  $Q \neq \emptyset$  do
9:    $(u, g) \leftarrow Q.dequeue()$ 
10:  iterations  $\leftarrow$  iterations + 1
11:  maxDepth  $\leftarrow \max(\text{maxDepth}, g)$ 
12:   $p \leftarrow \mathcal{T}.pred(u)$ ,  $s \leftarrow \mathcal{T}.succ(u)$ 
13:   $\Delta_{\text{rem}} \leftarrow d(p, s) - d(p, u) - d(u, s)$ 
14:   $\mathcal{N}_u \leftarrow \text{kNN}(\mathcal{K}, u, M)$ 
15:  bestGain  $\leftarrow 0$ , bestEdge  $\leftarrow \text{null}$ 
16:  for all  $v \in \mathcal{N}_u$  do
17:    for all edge  $(a, b) \in \{(v, \text{succ}(v)), (\text{pred}(v), v)\}$  do
18:      if  $a = u \vee b = u \vee a = p$  then
19:        continue
20:      end if
21:       $\Delta_{\text{add}} \leftarrow d(a, u) + d(u, b) - d(a, b)$ 
22:      gain  $\leftarrow -(\Delta_{\text{rem}} + \Delta_{\text{add}})$ 
23:      if gain > bestGain then
24:        bestGain  $\leftarrow$  gain
25:        bestEdge  $\leftarrow (a, b)$ 
26:      end if
27:    end for
28:  end for
29:  if bestGain >  $\epsilon \wedge \text{bestEdge} \neq \text{null}$  then
30:     $(a^*, b^*) \leftarrow \text{bestEdge}$ 
31:     $\mathcal{T}.moveNodeAfter(u, a^*)$   $\{O(1)$  pointer rewiring $\}$ 
32:    relocatedCount  $\leftarrow$  relocatedCount + 1
33:    if  $g + 1 \leq g_{\max}$  then
34:      for all affected  $\in \{p, s, a^*, b^*\} \cup \mathcal{N}_u$  do
35:        if affected  $\notin$  visited then
36:          visited.add(affected)
37:           $Q.enqueue((\text{affected}, g + 1))$ 
38:        end if
39:      end for
40:    end if
41:  end if
42: end while
43: return {iterations, maxDepth, relocatedCount}
```
